Stage C task plan: the size curve

Validated 8 November 2026. Gabriel Dupuis. Follows section 6.3 and section 8 of the revised project plan. Work started the same day: see the execution note at the end.

1. Goal and deliverable

Question. What is the smallest PubChem corpus at which descriptor pretraining still beats Chemprop trained from scratch?

Deliverable for Check-in 2 (week of 17 November). One figure: downstream score against pretraining corpus size, on BBBP and ESOL, with error bars from two seeds, the scratch Chemprop baseline as a horizontal line, and the released 1M checkpoint as the top point. Plus the operating size for Stage D, read off that curve.

What is held fixed. CheMeleon's recipe exactly as in the original implementation (hidden 2048, depth 6, batch 128, masking 0.15, early stopping with patience 50, max 500 epochs). Only the corpus size varies.

sizes	**500, 1k, 5k, 10k, 25k, 50k, 100k**
seeds	0, 1
corpus	PubChem, random draw from the 944,286 cached molecules
conditions	14 pretraining runs
evaluation	BBBP and ESOL, frozen scaffold splits, 5 seeds each, already on disk

2. What has to exist before the first GPU run

Three blockers, all on the critical path. None needs a GPU.

Blocker	**Why it blocks Stage C**	**Action**	**Owner, time**
B1. Frozen descriptor columns. Mordred leaves some columns undefined; 70 columns are all-NaN at n=500 and usable at n=100k, so the objective would grow with the corpus.	A rising curve could mean more data or a richer target, and the two could not be separated afterwards.	Compute the 100k store first, take its column set and standardisation vectors, reuse them unchanged at every smaller size. Record the frozen column count in the protocol.	G, half a day
B2. Fine-tuning loop. The fine-tuning script is a stub that stops with "not written yet".	Pretraining produces checkpoints; without this, no number comes out of any of them.	Write it against the protocol: 50 epochs max, patience 10, batch 64, Adam 1e-4, no tuning, one model per seed, one predictions file per run. Test on the released checkpoint and from scratch before our checkpoints exist.	L, 2 days
B3. Checkpoint and resume. The reference script starts training with no resume path. A 6-hour wall kill loses the run.	50k and 100k cannot finish inside 6 hours.	Wrap, do not edit the reference: resume from the last checkpoint when one exists, save it every 15 minutes, submit with requeue. Verify on the pilot by killing it at 10 minutes and resuming.	G, 1 day

One decision is already made by the code and should be confirmed rather than reopened: `scripts/pretrain.py` drives the **original** implementation (`external/chemeleon`, commit b5e5bd8). That is the only choice under which the released checkpoint is a comparable top of the curve. The rewrite is faster to train but is not

the same recipe.

3. Tasks, in order

#	Task	Depends on	Owner	Est.	Output
1	Pilot: pretrain PubChem n=500 seed 0 on one L40S, time it	nothing, store exists	G	1 GPU-h, 1 day of queue	seconds per epoch, epochs to convergence, memory
2	Recompute the budget table (section 4) from the pilot	1	G	1 h	go or pivot decision
3	Prepare the 14 corpora and the 100k descriptor store; freeze columns (B1)	nothing	G	one CPU job, 3 h	processed/corpora/, processed/descriptors/
4	Descriptor stores for the other 13 conditions with the frozen columns	3	G	one CPU job, 3 h	14 Zarr stores, 2.5 GB
5	Checkpoint and resume wrapper (B3), verified on the pilot	1	G	1 day	slurm/pretrain.sbatch with --requeue
6	Fine-tuning loop (B2), verified on released checkpoint and scratch	nothing	L	2 days	scripts/finetune.py, first BBBP and ESOL numbers
7	Scratch Chemprop baseline, both tasks, 5 split seeds	6	L	fine-tuning only	the horizontal line
8	Released 1M checkpoint fine-tuned, both tasks, 5 split seeds	6	L	fine-tuning only	the top point
9	Pretrain 500, 1k, 5k, 10k at both seeds	2, 4, 5	G	about 10 GPU-h, all inside 6 h each	8 checkpoints
10	Pretrain 25k at both seeds	9	G	about 20 to 35 GPU-h, 2 to 3 chunks each	2 checkpoints
11	Pretrain 50k at both seeds	10	G	about 35 to 65 GPU-h, preemptable	2 checkpoints
12	Pretrain 100k at both seeds	11, gate G3	G	about 65 to 130 GPU-h, preemptable	2 checkpoints
13	Fine-tune every checkpoint as it lands, both tasks, 5 split seeds	6, each of 9 to 12	L	fine-tuning only	predictions.csv per condition

14	Sensitivity: rerun the largest completed size with --exclude-benchmarks. Benchmark exposure grows along the curve, from one structure at 500 molecules to a few hundred at 100k, so the check belongs where it is largest, not where it is cheapest	11 or 12	G	5 to 30 GPU-h by size	exposure check
15	Curve figure, paired differences with intervals, operating size for Stage D	13	both	1 day	the Check-in 2 figure

Tasks 3, 4, 6, 7, 8 run in parallel with the pilot and need no GPU. By the time the pilot is timed, everything downstream of it should be ready.

---

4. Compute: what we have and what this costs

What the cluster gives us, checked on the login node on 8 November

partition	wall	per-user cap	GPUs	queue depth now
mit_normal_gpu	6 h	2 GPUs, 32 cores, 515 GB	212 L40S, 104 H200, 96 B200	1,085 pending, 201 running
mit_preemptable	48 h	none found	A100, H100	2,580 pending

Two things the plan did not know. **Preemption is instant**: the partition reports zero grace time and requeues the job, so the checkpoint interval is the amount of work at risk. And **the queues are deep**: over a thousand jobs ahead of us on the GPU partition this afternoon. Calendar time, not GPU-hours, is the binding constraint.

Two accounts give the team four concurrent GPUs on the normal partition instead of two. Both of us submit.

Cost per condition, to be replaced by the pilot's measurement

Plan's estimate: 1,000 GPU-hours on eight 2080 Ti for 1M molecules, L40S about three times faster per card, so 330 GPU-h per million molecules, with a factor of two penalty because small corpora converge in more epochs.

size	linear	with 2x penalty	6-hour chunks	both seeds
500 to 5k	under 2 GPU-h each	under 4	1	about 10 GPU-h total
10k	3	7	1 to 2	14
25k	8	17	2 to 3	34
50k	17	33	3 to 6	66
100k	33	66	6 to 11	132
total				about 130 to 250 GPU-h

The 100k condition alone is half the budget. That is where the pivots act.

Free speedups to take regardless

- `bf16-mixed` precision. The reference trains in fp32 because the 2080 Ti had

no bf16. Roughly 1.5 to 2x on an L40S, no change to the recipe's mathematics beyond rounding. State it.

- Dataloader workers raised from 4 to 12. Featurisation is the bottleneck;

CPUs are not scarce.

- Request L40S explicitly. Never H200 or B200; those queues are the contended ones

and the model fits in 44 GB many times over.

---

5. Pivot rules: decided now, triggered by numbers

Each gate has a date, a measurement, and a pre-agreed action, so we do not renegotiate the experiment under time pressure.

G1, 12 November: the pilot. Measured seconds per epoch at n=500.

- If a 100k run projects under 30 GPU-h per seed: proceed with the full table.
- If 30 to 60 GPU-h: proceed, but 100k runs on preemptable only and starts the

same day as 25k, not after it.

- If over 60 GPU-h: **Pivot A**, drop 100k. The curve runs 500 to 50k, and the

released 1M checkpoint remains the top point. The question "where does it turn" is still answered if it turns below 50k, which is where Yuxin expects it.

G2, 14 November: the small sizes. All of 500 to 10k done at both seeds.

- If not done because of queue time rather than compute: **Pivot B**, drop seed

1 at 50k and 100k, keep two seeds at 25k and below. Report the large sizes with one seed and say so.

- If epochs to convergence at 10k exceed 300: the 2x penalty was optimistic.

Recompute G1 and apply its rule.

G3, 17 November, Check-in 2. Curve in hand to 25k or 50k, baseline and released checkpoint scored.

- Decide whether to start 100k at all. It only runs if it can finish by

21 November, since Stage D needs the GPUs from then on.

G4, 21 November: hard stop for Stage C pretraining. Whatever has finished is the curve. Unfinished conditions are reported as unfinished, with the epochs reached, not extrapolated.

Pivot C, if the queue blocks everything, meaning no run has started by 14 November: switch to the plan's fallback, continued pretraining from the released checkpoint on each corpus subset. This is a weaker experiment and the write-up says so in the first paragraph: it measures what the corpus does to a model that already saw a million PubChem molecules, not what the corpus teaches from nothing. It costs one tenth of the GPU-hours and fits in single 6-hour jobs.

Pivot D, if the recipe is the problem. If early stopping at patience 50 is what makes the large runs long, cap epochs at 100 with patience 10 **for every size** and rerun the small sizes under the same cap. Uniform cap across the curve keeps the comparison fair; the released checkpoint then stops being strictly comparable and is drawn as a reference line rather than a curve point.

What we do not do under any pivot: tune per condition, compare across different splits, mix the two implementations, or drop the scratch baseline.

---

6. Schedule

when	what	who
Mon 10 Nov	Submit pilot. Prepare corpora and 100k store. Start fine-tuning loop.	G, G, L

Tue 11	Resume wrapper written and tested on the pilot. Freeze columns.	G
Wed 12	G1. Budget table recomputed. Submit 500 to 10k, both seeds. Fine-tuning loop first numbers.	G, L
Thu 13	Scratch baseline and released checkpoint scored. 25k submitted.	L, G
Fri 14	G2. Small sizes fine-tuned. 50k submitted on preemptable.	both
Mon 17	G3, Check-in 2. Curve to 25k or 50k. Decide 100k.	both
Tue 18 to Fri 21	50k and 100k land and are fine-tuned. Sensitivity run.	both
Fri 21	G4. Stage C closes. Operating size for Stage D fixed.	both

7. What I need validated

1. The three blockers in section 2 and their owners.
2. The gate thresholds in G1: 30 and 60 GPU-hours per seed as the cut points.
3. Pivot A before Pivot B: dropping 100k before dropping a seed. The alternative ordering is defensible if Yuxin cares more about the top of the curve than about error bars.
4. bf16-mixed precision as a departure from the reference that we accept.
5. 21 November as the hard stop for Stage C GPU work.

8. What is already in place

- 944,286 cached PubChem molecules, canonicalised and deduplicated.
- The n=500 seed 0 corpus and its descriptor store, 1,613 features, built and checked.
- Frozen scaffold splits for BBBP and ESOL, five seeds each: BBBP 1,631 / 203 / 205 after dropping 11 unparseable rows, ESOL 902 / 112 / 114.
- Released CheMeleon checkpoint downloaded and verified.
- Both CheMeleon implementations vendored and pinned.
- Job scripts that skip finished steps and survive preemption, for the CPU side.
- Stage C configuration file with sizes and seeds as above.

9. Execution note, 8 November

Started on validation. Three things changed on contact with the code.

B1 is done, and it uncovered a defect that would have broken pretraining. `compute_descriptors.py --finalize` now takes `--reference`, which reuses another condition's standardisation vectors instead of computing its own, so the objective stays fixed while the corpus grows. While testing it, the 500-molecule store turned out to carry 10 descriptors with an undefined variance and 117 with zero variance, on top of the 70 that are undefined everywhere. Dividing a target by either would have produced NaN or infinite targets on the first pretraining run. All three classes are now neutralised to mean 0 and variance 1, which passes them through untouched, and the counts are recorded per condition. The usable-target count this produces, 1,416 of 1,613 at n=500, matches `docs/open-issues.md` exactly.

The pretraining job would have broken every CPU job. It ran `uv sync --extra train`, which is exact

and therefore uninstalls the descriptor stack the whole Stage A pipeline needs. The training stack now lives in its own environment, `.venv-train`, so a GPU job and a CPU job can never fight over one set of packages. The job also requests an L40S through the GRES rather than a node feature, which does not exist on this cluster.

Task 3 and task 4 merged into `slurm/stage-c-prep.sbatch`: it draws all 14 conditions, describes the 100,000-molecule reference first, freezes the vectors against it, then describes the other thirteen against that reference. One queue wait instead of two, and every step skips work already on disk.

---

10. Gate G1, measured 8 November

The pilot ran: PubChem 500 molecules, seed 0, one L40S, fp32. It trained for 329 seconds over 308 epochs, early-stopping on a best epoch of 258, and reached a test mean squared error of 0.305 on the standardised descriptor targets.

Measured throughput: 364 training molecules per second, 0.96 seconds per epoch at three batches of 128. Projecting it across the curve, with the pilot's own 308 epochs as the low case and the 500-epoch cap as the high one:

size	train molecules	at 300 epochs	at 500 epochs	6-hour chunks
500	350	0.1 h	0.1 h	1
1,000	700	0.2 h	0.3 h	1
5,000	3,500	0.8 h	1.3 h	1
10,000	7,000	1.6 h	2.7 h	1
25,000	17,500	4.0 h	6.7 h	1 to 2
50,000	35,000	8.0 h	13.4 h	2 to 3
100,000	70,000	16.0 h	26.7 h	3 to 5
both seeds		61 GPU-h	102 GPU-h	

G1 passes, and comfortably. The rule was to proceed with the full table if a 100,000-molecule run projects under 30 GPU-hours per seed. It projects 16 at the pilot's epoch count and 27 at the 500-epoch cap, so the full curve runs as planned. The whole of Stage C costs 61 to 102 GPU-hours against the plan's estimate of 130 to 250, because the project plan assumed an L40S is three times a 2080 Ti while the measured gap is wider.

Three consequences.

- **Only 50,000 and 100,000 need chunking.** Everything to 25,000 fits in one

six-hour job, which is eleven of the fourteen conditions. The checkpoint-and-resume wrapper is still required, but it is now needed for three conditions rather than for the whole curve.

- **bf16 is upside we have not spent.** The reference trains in fp32 because

the 2080 Ti had no bf16. The L40S does, and the dataloader runs four workers against sixteen requested cores, so there is headroom on both sides.

- **Preemption is real.** The preparation job was preempted twice in

twenty-five minutes on `mit_preemptable` and requeued itself both times, losing only the work since its last completed descriptor shard. For a 16-hour pretraining run that behaviour would be ruinous without checkpointing, which is the argument for B3 made from measurement rather than from caution.

Cost per condition on disk, corrected after B3 landed: three checkpoints of 143 MB rather than two, because the wrapper's `last.ckpt` sits alongside the recipe's own two best-by-val_loss files, plus a 50 MB exported model. That is 478 MB a condition and 6.7 GB for the full curve, against 4.7 GB before. Home holds 15.6 GB of a 200 GB quota, so the margin is not in question. Peak memory 3.1 GB against the 64 GB requested, which should come down.

11. Blockers B1 and B3 cleared, 8 November

B1, the frozen descriptor objective. `compute_descriptors.py --finalize` now takes `--reference`, so the thirteen smaller conditions standardise against `pubchem_n100000_s0` instead of against themselves. Without it the curve would have confounded corpus size with the width of the regression objective, since which Mordred descriptors come out defined depends on the sample. Measured on the 500-molecule condition, 70 of the 1,613 descriptors are undefined everywhere, 10 are defined for exactly one molecule and 117 are constant; the first two give an undefined variance and the third gives zero, so dividing targets by them yields NaN or infinity. Those columns are neutralised to mean 0 and variance 1, which passes them through untouched, and the counts are recorded so the objective's real width is known rather than assumed.

Those three counts are what the 500-molecule condition would give if it were standardised against itself, leaving 1,416 usable targets, and they are the argument for not doing that. The frozen objective, measured once the preparation job had finalised all fourteen conditions against `pubchem_n100000_s0`, is **1,562 usable targets of Mordred's 1,613** - the width every point on the curve now trains against, from 500 molecules to 100,000. Verified rather than assumed: all fourteen conditions carry byte-identical means and variances, every one records `standardised_against: pubchem_n100000_s0`, and no variance is zero or non-finite. Had each condition standardised against itself, the smallest would have fit 1,416 targets and the largest 1,562, and the curve would have measured the objective widening at the same time as the corpus growing.

B3, resumption. Verified end to end rather than argued for. Job 28371249 ran the pilot with a three-minute wall and a 30-second checkpoint interval and was killed by Slurm at epoch 107, leaving `last.ckpt` beside the recipe's own two checkpoints - which is the practical confirmation that patching `Trainer.__init__` appends to the recipe's callback list rather than replacing it. Job 28372537 then resubmitted the same condition with no override and logged `resuming from .../last.ckpt`, with its first epoch numbered 107 and not

1. The epoch number is the whole test: a run that resumed but silently

restarted from zero would be indistinguishable by wall-clock on a 500-molecule condition and would quietly cost sixteen hours on a 100,000-molecule one.

The interval is a timer rather than an epoch count for two reasons. Lightning treats the two as mutually exclusive, and an epoch is the wrong unit across a size curve: one epoch of the 500-molecule condition takes about a second, which would write 143 MB to shared storage every second, while one epoch of the 100,000-molecule condition takes roughly three minutes. A fixed interval bounds both the work at risk and the write rate at every point on the curve. `NC_CHECKPOINT_MINUTES` overrides the 15-minute default, which is the only reason the kill test could run inside three minutes.

What resumption restores is the optimiser, the scheduler, the epoch counter and the early-stopping state, so a resumed run follows the trajectory of an uninterrupted one. What it does not restore is the dataloader's shuffle position inside the interrupted epoch, so that epoch replays from its start. At a 15-minute interval that is a fraction of one epoch on any condition here.

Scheduling, revised on measurement. The preparation job asked for 32 cores and 12 hours and was scheduled for 9 November at 18:00. The scheduler estimated 10 November at 14:07 for that shape on `mit_normal` - and the same estimate for every smaller shape, down to 4 cores and 2 hours, so the request shape was not the lever. `mit_preemptable` estimated the same evening at 21:10 with 8 idle nodes. Preemption was the reason that partition was abandoned and is no longer an argument against it: each descriptor shard flips its own flag, so being killed costs one shard of at most 10,000 molecules, about four minutes. The job now asks for 16 cores and 4 hours across both partitions with `--requeue`, sized from the Stage A rate of roughly 45 molecules a second on 8 cores, which puts the whole 383,000-molecule curve near 2.4 CPU-hours.

Outcome. Job 28373122 started the moment it was submitted rather than at the estimated 21:10, and

finished all fourteen conditions in 1:30:13 with zero failed steps. It reused the eight descriptor shards an earlier preempted job had already written, which is the shard-level idempotency paying for itself; that reuse was checked rather than trusted, by recomputing Mordred for two molecules inside completed shards and confirming they reproduce the stored rows exactly. Describing scaled as expected with size: the 100,000-molecule conditions took about 9 and 29 minutes, the 50,000 ones 14 minutes each, and everything at 10,000 and below finished in under three minutes apiece.

12. First wave and gate G1 recomputed, 8 November

The eight small conditions were submitted at 20:28. By 21:45 five had finished and three were still training, none preempted.

condition	stopped at epoch	test MSE	wall time
500, seed 0	227	0.296	6 min
500, seed 1	82	0.448	2 min
1,000, seed 0	241	0.310	13 min
1,000, seed 1	164	0.337	9 min
5,000, seed 0	308	0.149	68 min
5,000, seed 1	at 270, running		
10,000, both seeds	at 131 and 137, running		

Every finished run stopped exactly 50 epochs after its best validation epoch, exported a 50 MB `best.pt`, and none reached the 500-epoch cap. The test MSE is on standardised descriptors over each condition's own held-out tenth, so it is a health check and not a curve: the test sets differ between conditions, and at 500 molecules the test set is 50 molecules, which is why the two seeds sit 0.15 apart. The curve's y-axis is fine-tuned BBBP and ESOL performance, which needs the fine-tuning loop.

Throughput is lower than the pilot said. Gate G1 was passed on 364 training molecules a second, measured on the 500-molecule pilot. End to end, validation included, the real runs deliver 269 (5,000, seed 0), 214 (5,000, seed 1) and about 212 (10,000, both seeds). The cause is measured, not guessed: eight one-second samples of GPU utilisation on a running job read 0% seven times and 84% once. The L40S is idle most of the time and the run is limited by the reference script's four dataloader workers, so a faster GPU would change nothing and the rate should hold at every size.

At 215 to 270 molecules a second over the 70% training split:

size	per epoch	300 epochs	500-epoch cap
25,000	65 to 81 s	5.4 to 6.8 h	9 to 11 h
50,000	130 to 163 s	11 to 14 h	18 to 23 h
100,000	259 to 326 s	22 to 27 h	36 to 45 h

A 100,000-molecule run therefore projects 22 to 45 GPU-hours a seed, not 16 to

1. That straddles the 30 GPU-hour line of gate G1, whose pre-agreed rule for

the 30-to-60 band is: proceed, 100k on preemptable only, started the same day as 25k rather than after it. Applied as written. The six large conditions were submitted at 21:46: 50k and 100k on `mit_preemptable` with a 48-hour wall and `--requeue`, and 25k on `mit_normal_gpu` as chains of three 6-hour jobs, each link starting only if the previous one did not finish and resuming from its `last.ckpt`. The remaining six cost 76 to 158 GPU-hours depending on where early stopping fires, against the 61 to 102 quoted for the whole curve in section 10, which is superseded.

The per-user caps bound the calendar: 2 GPUs on `mit_normal_gpu` and 4 on `mit_preemptable`. With the worst case of 45 hours for 100k, every condition finishes by 11 November even if early stopping never fires, ten days inside the 21 November hard stop of gate G4.

One option left open. Because the GPU is idle most of the time, raising the dataloader worker count from the wrapper could plausibly halve the wall time without touching the model, the optimiser or the data order. It is

not applied: the runs fit the schedule without it, it would need its own check that the masking stays statistically identical, and changing how a run is loaded half-way through the curve is a difference a reader would have to be talked out of.